

VendorHub AI

“Find the Right Supplier. Faster. Smarter.”

Team Task Division Plan — Final Year Project

Team Member	Role	Work Package
Muhammad Khan	AI Engineer & Project Integration Lead	Part 1 – AI Systems & Integration
SaqlainAliSajid	Frontend Developer	Part 2 – Buyer-Side Frontend
UMAIR ALI	Frontend Developer	Part 3 – Vendor & Admin Frontend
Medo_Haytham	Backend Developer	Part 4 – Core Backend & APIs
Youssef	Backend Developer	Part 5 – Real-Time, Orders & Integrations

Note: Team assignments above are a suggested split based on the group list. Names can be swapped between Frontend/Backend roles as the team prefers — the module breakdown stays the same.

Project Overview

VendorHub AI is an AI-powered B2B sourcing platform that connects businesses with verified suppliers, manufacturers, wholesalers, and service providers worldwide. Users describe what they need in natural language, and the AI identifies suitable vendors, compares them, estimates costs, generates RFQs (Request for Quotations), and manages communication — all from one platform.

Core Tech Stack

Layer	Technologies
Frontend	Next.js, React, TypeScript, Tailwind CSS
Backend	FastAPI (Python), Node.js (real-time chat)
Database	PostgreSQL, Redis, Elasticsearch
AI	OpenAI GPT, Gemini, OCR Engine, Translation API
Cloud	AWS / Azure / Google Cloud

The project is split into 5 work packages below. Each package lists the SRS modules it owns, concrete deliverables, and the tech it touches — so every member can work independently and Muhammad Khan integrates all parts into the final system.

Part 1 — AI Systems & Project Integration

AI Engineering & Integration Lead

Assigned to: Muhammad Khan

Responsibility

Own every AI capability in the platform and integrate the Frontend and Backend work packages into one working product — connecting APIs, resolving conflicts, and managing deployment.

SRS Modules Owned

- Module 4 — AI Supplier Search (natural-language query understanding + filtering)
- Module 7 — AI Vendor Matching (match-score engine)
- Module 8 — RFQ Generator (AI-drafted RFQs)
- Module 9 — Quote Comparison (AI recommendation logic)
- Module 10 — AI Negotiation Assistant (emails, counter-offers, follow-ups)
- Module 13 — AI Risk Analysis (supplier risk scoring)
- Module 14 — Documents (AI summarization & search)
- Cross-cutting: demand forecasting, product recommendations, auto-translation, procurement insights

Deliverables

- AI microservice(s) exposing REST endpoints (e.g. /search, /match-score, /generate-rfq, /compare-quotes, /negotiate, /risk-score)
- Prompt-engineered pipelines using OpenAI GPT / Gemini for each AI feature
- OCR + translation API integration for documents and messaging
- Integration layer connecting Frontend (Parts 2–3) with Backend APIs (Parts 4–5)
- End-to-end testing across all 5 parts before final submission
- Final deployment (AWS / Azure / GCP) and environment configuration

Tech Stack

Python, FastAPI, OpenAI GPT API, Gemini API, LangChain (optional), OCR engine, Translation API, Docker, cloud deployment tools.

Part 2 — Buyer-Side Frontend

Frontend Developer

Assigned to: SaqlainAliSajid

Responsibility

Build every screen a Buyer sees — from login to placing and tracking an order.

SRS Modules Owned

- Module 1 — Authentication (Login, Signup, Google/LinkedIn/Microsoft login, email verification, 2FA, forgot password)
- Module 2 — Buyer Dashboard (Active RFQs, Pending Quotes, Orders, Saved Vendors, AI Recommendations, Recent Searches, Spending Summary)
- Module 4 (UI) — AI Supplier Search bar & filters (country, industry, MOQ, price, certifications, lead time)
- Module 5 — Vendor Profile pages (company info, products, photos/videos, certifications, reviews, ratings)
- Module 6 — Product Catalog (categories, search, images, specs, MOQ, price range)
- Module 15 — Reviews & Ratings (buyer review submission UI)

Deliverables

- Responsive Buyer web pages (Next.js + React + TypeScript + Tailwind CSS)
- Reusable component library shared with Part 3 (buttons, cards, modals, forms, nav bar)
- Vendor profile & product catalog UI wired to backend product/vendor APIs
- Buyer dashboard widgets consuming backend/AI endpoints

Tech Stack

Next.js, React, TypeScript, Tailwind CSS, React Query / Axios for API calls.

Part 3 — Vendor & Admin Frontend

Frontend Developer

Assigned to: UMAIR ALI

Responsibility

Build every screen a Vendor and an Admin see — profile management, RFQ handling, and platform administration.

SRS Modules Owned

- Module 3 — Vendor Dashboard (New RFQs, Active Orders, Revenue, Customer Requests, Product Performance, Analytics)
- Module 8 (UI) — RFQ Generator forms (product, quantity, material, budget, delivery date, payment terms, PDF export)
- Module 9 (UI) — Quote Comparison table (Vendor A/B/C feature grid)
- Module 10 (UI) — AI Negotiation Assistant chat/compose screen
- Module 11 — Messaging (chat, file sharing, voice notes, image sharing, read receipts, translation UI)
- Module 16 — Notifications (RFQ received, new quote, order shipped, message, contract expiring, payment reminder)
- Admin panel — manage users, verify vendors, categories, reports, subscription plans, fraud monitoring

Deliverables

- Vendor dashboard and profile-management UI (Next.js + React + TypeScript + Tailwind CSS)
- RFQ creation form and quote-comparison screens
- Real-time messaging UI wired to the Node.js chat backend (Part 5)
- Notification center + Admin dashboard for platform management

Tech Stack

Next.js, React, TypeScript, Tailwind CSS, WebSocket client for chat/notifications.

Part 4 — Core Backend & APIs

Backend Developer

Assigned to: Medo_Haytham

Responsibility

Build the core data layer and REST APIs that power authentication, dashboards, vendor profiles, the product catalog, and reviews.

SRS Modules Owned

- Module 1 — Authentication backend (JWT/session auth, OAuth for Google/LinkedIn/Microsoft, email verification, 2FA, password reset)
- Module 2 & 3 (data) — Buyer & Vendor Dashboard APIs (aggregation queries for widgets)
- Module 5 — Vendor Profile CRUD APIs (company info, certifications, media uploads)
- Module 6 — Product Catalog APIs (categories, search, filters, stock)
- Module 15 — Reviews & Ratings APIs
- Database schema design & migrations (PostgreSQL) for all of the above

Deliverables

- FastAPI service with documented REST endpoints (OpenAPI/Swagger) for auth, users, vendors, products, reviews
- PostgreSQL schema + migrations, Elasticsearch indexing for product/vendor search
- Role-based access control (Buyer / Vendor / Admin)
- Unit tests for all endpoints, ready for the AI layer (Part 1) and Frontend (Parts 2–3) to consume

Tech Stack

FastAPI (Python), PostgreSQL, Elasticsearch, JWT/OAuth2, Alembic (migrations).

Part 5 — Real-Time, Orders & Integrations

Backend Developer

Assigned to: Youssef

Responsibility

Build everything that happens after an RFQ is sent: real-time messaging, order/shipment tracking, notifications, documents, and third-party integrations.

SRS Modules Owned

- Module 11 — Messaging backend (Node.js real-time chat, file/image/voice-note sharing, read receipts)
- Module 12 — Order Management (purchase orders, status tracking, delivery updates, shipment timeline, invoices, payments)
- Module 14 — Document storage APIs (upload/retrieve contracts, certifications, POs, invoices, shipping docs)
- Module 16 (backend) — Notification service (push/email triggers for RFQs, quotes, orders, messages, contracts, payments)
- Module 17 — Analytics APIs (spending, top suppliers, cost savings, revenue, RFQ conversion rate)
- Payment & logistics integrations: Stripe, PayPal, Wise, DHL, FedEx, UPS

Deliverables

- Node.js real-time chat service (Socket.io/WebSocket) integrated with Redis for presence/queuing
- Order management REST API with status/shipment tracking
- Payment gateway integration (Stripe/PayPal sandbox) and shipping-carrier API integration
- Notification service (email + in-app) and analytics endpoints for dashboards

Tech Stack

Node.js, Express.js, Socket.io, Redis, Stripe/PayPal SDKs, DHL/FedEx/UPS APIs.

How the Parts Connect

All backend APIs (Parts 4 & 5) should follow one shared API contract from day one, agreed on by the whole team, so the AI layer (Part 1) and both frontend teams (Parts 2 & 3) can build against it without waiting.

- Frontend (Parts 2 & 3) → calls → Backend REST APIs (Parts 4 & 5) for data (auth, vendors, orders, chat, etc.)
- Frontend (Parts 2 & 3) → calls → AI endpoints (Part 1) for search, matching, RFQ generation, negotiation, risk score
- AI layer (Part 1) → reads/writes → Backend database via Parts 4 & 5 APIs (e.g. saving generated RFQs, fetching vendor data to score)
- Muhammad Khan integrates all four streams, resolves API mismatches, and handles final deployment

Suggested Working Order

- Week 1–2: Agree on API contract & database schema (all 5 members). Parts 4 & 5 start building auth + core tables.
- Week 3–5: Parts 2 & 3 build UI against mock/early APIs. Part 1 builds AI prototypes with sample data.
- Week 6–8: Wire real APIs into frontend. Part 1 connects AI endpoints to backend data.
- Week 9–10: Muhammad Khan integrates everything, full end-to-end testing, bug fixing, deployment.